Data-Driven Design & Analysis of Structures & Materials (3dasm)

Instructor: Miguel A. Bessa

Brown University

Homework 7

Deliver a **ZIP folder with two files (PDF and Jupyter notebook)** containing the answers to the questions of this assignment. **UPLOAD to CANVAS in the Assignments section (Homework 7) by the due date.**

Due date: until 11:59pm of day announced in course home page.

This Homework aims to help you reason about the design of experiments and reproducibility when AI agents carry out the data-driven process, using `adda` to define the agents, their tools and their teams. This is a homework solved with **code** and agents: the notebook you submit contains your problem statements, your tools and agents, their results and your answers. Exercise 2 is also the first step of the Final Project: it practices the table of decisions, the team and the checks that the project asks for.

How to work. Start from the notebook `Assignments/3dasm_Homework7.ipynb`: copy it to `Assignments/your_Assignments/HW7/your_Homework7.ipynb` and work there. The agents work in a study folder, and the notebook is the source of truth: each problem statement, tool, agent and team is a notebook cell, and the cells that we provide create the study folders.

- **Setup.** Both exercises need `adda`, installed as in Lecture 19, and Claude Code, logged in with your course account. They run on your computer, not on Colab. Use Haiku for every agent (`model="haiku"`). A run of this homework takes a few minutes; expect about five runs in total.
- **No instructions file.** `adda` does not read an instructions file (`CLAUDE.md`): what you wrote there in Homeworks 4 to 6 now goes in the problem statement (the task and every decision), in each agent's `system_prompt` (its role and rules), and in its `tools` (what it can do: the only part that is enforced).
- **A study runs once.** The cells we provide keep each study's report and run it again only when its statement changes. If a run fails, delete its study folder before running again.
- **Grading.** The TA grades your submitted notebook and study folders; nothing is run again. In Exercise 1 the agent chooses where to evaluate the function, so its numbers are not graded: your tool, your statement, your predictions and your checks are. In Exercise 2 your final statement fixes every decision, so your two runs must return the same numbers. Base your answers on the files and figures your notebook stores, not on the agent's report.
- **Your problem statements and your tools are graded.** A good statement defines the problem **uniquely**, as the prompts of Homeworks 4 to 6 did.
- **Write your statements in your own words.** A statement that only points to a question (e.g. "solve exercise 3.3") or that pastes it is not a statement: that item counts as not answered. You may copy the specifications (numbers, seeds, names of functions).
- The checks that a question asks you to do yourself are your own code, in a notebook cell, not an agent's.
- **Submit one ZIP file** with your notebook (`.ipynb`), a PDF of it, and the study folders `hw7_ex1`, `hw7_vague`, `hw7_pinned_run1` and `hw7_pinned_run2`.

Exercise 1 (with the agent)

In this exercise an agent searches for the minimum of an expensive function through a tool that you write, and the study can evaluate the function at most 20 times. For $\mathbf{u} = (u_1, u_2) \in [0,1]^2$, let $x_1 = -5 + 15u_1$ and

$x_2 = 15u_2$, and

$$g(\mathbf{u}) = \frac{f(x_1, x_2) - 0.397887}{10} + 2, \qquad f(x_1, x_2) = \left(x_2 - \tfrac{5.1}{4\pi^2}x_1^2 + \tfrac{5}{\pi}x_1 - 6\right)^2 + 10\left(1 - \tfrac{1}{8\pi}\right)\cos x_1 + 10.$$

The minimum value is $g^\star = 2$, at the three points that correspond to $(x_1, x_2) = (-\pi, 12.275)$, $(\pi, 2.275)$ and $(9.42478, 2.475)$. The agent must find the minimum by calling the tool: the statement and the docstring do not give the expression of g. In 1.5 you compare the agent with **Latin Hypercube sampling**: for N points, it divides the range of each input into N sub-intervals of equal length and draws one point uniformly inside each sub-interval, and the sub-intervals of the different inputs are paired by a random permutation, so that every sub-interval of every input holds exactly one point.

1. Minimizing an expensive function with an agent.

   1.1. Write a Python function `Measure(u1, u2)` that returns $g(u_1, u_2)$, refuses any point outside $[0, 1]^2$ and every call after the 20th, and records every accepted call. Its docstring is all the agent is told about it. After the run, store the record as an `ExperimentData` of `f3dasm` in the study folder.

   1.2. Write the problem statement: find the minimum of the function behind `Measure`, and write the best point and its value in `best.json`. Use `required_deliverables` so that the agent cannot call `Done` without `best.json`.

   1.3. **Before running anything**, write down your predictions: how will the agent spend its 20 evaluations, and how close to $g^\star = 2$ will its best value be? (You will not lose points for wrong predictions. You will lose points for not writing them.)

   1.4. Run the agent once. Show a contour plot of g on $[0, 1]^2$ with the points of your record, numbered in the order of the calls, and the best point of `best.json`.

   1.5. **Check** the agent's result: is the point of `best.json` in your record, with the same value, and is it the best point of your record? How far is it from the nearest of the three minima? Compare with 20 points of Latin Hypercube sampling (`create_sampler('latin_sampler', seed=0)`): report their best value, and compare how its points and the agent's cover the square.

   1.6. **What can you conclude?** Compare with your predictions of 1.3.. Open the files of the run, as in Lecture 19: is the code that produced `best.json` in the study folder? Your tool limits the evaluations that go through it: can it stop the agent from computing g in another way, given that your notebook, where g is written, is a file the agent can read? How could you hide g from the agent? Which change to your statement or to your tool would help the agent most?

Exercise 2 (with the agent)

In this exercise you turn a vague problem statement into one that fixes every decision, so that two runs return the same numbers and an independent check agrees with them. The statement compares two optimizers of `scipy.optimize` that use no derivatives: **Nelder-Mead** moves a triangle of points (a simplex) downhill and stops when the simplex and its values change less than the tolerances `xatol` and `fatol`; **Powell** minimizes along one direction at a time and stops on the tolerances `xtol` and `ftol`. Both report the number of evaluations of the function they used, `nfev`. The test function is the 2-D Rosenbrock function, `scipy.optimize.rosen`, with its minimum 0 at $(1, 1)$.

2. A reproducible comparison of two optimizers.

   2.1. Run the problem statement *"Compare the Nelder-Mead and Powell optimizers of scipy on the 2-D Rosenbrock function, and say which one is better. Write the comparison in results.csv, next to this file."*, verbatim, twice, with one agent and `required_deliverables=["results.csv"]`. Give each run its own study folder inside its own parent folder (`hw7_vague/run1/study` and `hw7_vague/run2/study`).

   2.2. Do the two runs agree? Write the table of decisions that the statement leaves to the agent (at least six rows), and for each one say which reported number, or which conclusion, it can change. Then read the files that the agents wrote (their scripts and `results.csv`, not only their reports), and mark the decisions that the two runs made differently, with the file and the line that shows it.

2.3. Rewrite the statement as a table of decisions, so that only one implementation is possible: the function, the two optimizers and their options, the starting points and their seed, the number of repetitions, what counts as a success, the metrics, what "better" means, and the files to deliver (in `workspace/results/`) with their columns. Define a team with at least a builder and a checker; the checker recomputes every number without the builder's code and writes PASS or FAIL for each one in `workspace/results/check.md`. (Hint: give the full path of the study folder in the coordinator's prompt.)

2.4. Run your study twice, each run in its own study folder, and show that the two `results.csv` agree. Then recompute one number of `results.csv` yourself, in a notebook cell. If the runs disagree, or a check fails, correct the statement (not the results) and run again, and report each decision you added and why.

2.5. **What can you conclude?** Which optimizer is better with your decisions? Would it still be better for an engineer who needs the minimum of f to an accuracy of 10^{-12}? Which rows of your table decide it, and would changing any of them change the verdict? Are two identical runs enough to trust your results?